text

Beyond Ingress: Using cert-manager Across Your Cluster

Speaker

Matt Bates (Jetstack) — Original creators of cert-manager; now a CNCF project with contributions from ~280+ community members.

---

1) Overview

cert-manager is a Kubernetes add-on that extends the Kubernetes API with **CRDs** to manage **X.509/TLS certificates** and **issuers**. It automates the full lifecycle—**issuance, renewal, and distribution**—so applications can consume certificates declaratively, just like Pods or Deployments.

- **Adoption:** Widely used in production; donated to CNCF (Sandbox at the time referenced).
- **Ecosystem:** Built-in issuers and many **external issuers** (public & private PKI).

---

2) How cert-manager Works (Core Model)

- **CRDs:**
  - **Issuer/ClusterIssuer** — "Where/how" to get certificates (auth, endpoints, type).
  - **Certificate** — "What identity" (DNS/IP/URI SANs, usages, duration, key params).
  - **CertificateRequest** — Point-in-time request (encodes CSR) used by controllers.
- **Controller loop:** Reconciles desired state → obtains signed cert from your CA → stores it (by default) in a **Kubernetes Secret** (PEM, PKCS#12, JKS supported).
- **Common CA options:** Let's Encrypt (ACME), HashiCorp Vault, Venafi, Google CAS, AWS PCA, Smallstep, and more (including external issuers).

---

3) The Familiar Path: Ingress TLS (ACME)

For public HTTPS, cert-manager's **Ingress shim** watches annotated Ingress objects, creates the necessary **Certificate / CertificateRequest**, automates the **ACME** flow (Orders/Challenges), and writes the resulting cert/key to a **Secret** consumed by your Ingress controller (e.g., NGINX).

Why it's popular: Simple UX (annotate Ingress), automated renewals, publicly trusted certs.

4) Direct Use of Certificate Resources

Beyond Ingress, you can declare **Certificate** resources directly in Git (GitOps) for any workload that needs TLS:

- Specify **short lifetimes** and **usage** (e.g., `serverAuth`, `clientAuth`).
- Reference a **private issuer** for internal identities.
- Mount the produced Secret into Pods (volume + volumeMount) and serve TLS in the app.

Use cases: Internal services, database client auth, internal dashboards, cronjobs, operators, etc.

---

5) Private PKI Options (No Public CA Needed)

For **internal** identities (east-west traffic), use a **private CA**:

- **SelfSigned + CA Issuer** (built-in): Bootstrap a cluster-local CA quickly (mark a cert `isCA: true`, back a CA Issuer with it).
- **Vault / Smallstep / Cloud CAs**: Swap in more robust PKI backends.
- **Enterprise**: Venafi TPP issuer, AWS PCA, Google CAS, external issuers.

This lets you issue internal mTLS certs without touching public trust anchors.

---

6) Pod-to-Pod mTLS (without a Service Mesh)

When you have a **small set of microservices** and want **mutual TLS** without the operational overhead of a mesh:

- Create **two Certificates** (e.g., `ping` and `pong`) with appropriate **`usages`** (`serverAuth`, `clientAuth`).
- Use the **same issuer/CA** so both sides trust each other.
- Mount certs/keys as files; each app validates the peer's cert chain (same CA).

This pattern is simple, transparent, and fully automated by cert-manager's renewal loop.

---

7) Database Client Authentication

Use cert-manager to mint **client certificates** for DBs (e.g., MySQL, Postgres) and **server certificates** for the DB endpoints:

- Encode **clientAuth** in the `usages`.

- Mount cert/key into application Pods.
- Enforce **short lifetimes** + automated rotation for tighter security.

---

8) CSI Driver: Secretless, Node-Local Keys

The **cert-manager CSI Driver** provisions certs **as volumes** so **private keys remain node-local** and **in memory** (not persisted in Kubernetes Secrets):

How it works (DaemonSet on every node):

1. Pod schedules; kubelet calls NodePublishVolume.
2. Driver **generates private key** + **creates CertificateRequest**.
3. cert-manager gets the cert from your issuer.
4. Driver writes key/cert to the node FS and **bind mounts** into the Pod (memory-backed).
5. Driver **tracks and renews**; on Pod delete, kubelet triggers clean-up.

Benefits:

- No Secrets for keys; **per-Pod unique identity** (great for auditability).
- Works well for dynamic workloads needing short-lived certs.

---

9) Securing Admission Webhooks (CA Injector)

Kubernetes **admission webhooks** (validating/mutating) must be TLS-secured. cert-manager's **CA Injector** watches annotations and **injects CA bundles** into webhook configurations.

- Annotations like `cert-manager.io/inject-ca-from` (from a Certificate or Secret) or API server CA injection.
- cert-manager itself uses this model for its own webhook.

Example in the wild: Cluster API installs cert-manager and uses Certificates in the `capi-webhook-system` namespace for its webhooks.

---

10) Service Mesh Integrations

Service meshes provide **transparent mTLS**, policy, and observability. cert-manager can supply the **trust anchor** and/or **workload certs**:

- **Linkerd:** Use cert-manager to automate trust anchor and issuer; Linkerd automates short-lived leaf certs thereafter.

- **Istio:** Historically supported plugging a custom CA; ongoing work enables cert-manager to handle workload certs (including experiments with the Kubernetes **Certificates API**).
- **Open Service Mesh (OSM):** Pluggable cert management with support for cert-manager, Vault, Azure Key Vault, etc.

Result: Meshes get automated, policy-enforced identities, backed by the issuer you choose.

---

11) Control Plane & Node Certificates

11.1 Control Plane (kubeadm)

- **kubeadm** auto-generates and auto-renews control-plane certs.
- For enterprises needing existing chain of trust, use **External CA mode**: kubeadm generates CSRs; an external workflow (e.g., Venafi integration) signs them.
- `kubeadm certs generate-csr` is GA; can integrate with cert-manager-backed CA flows.

11.2 Kubelet / Node Certs (Kubernetes Certificates API)

- Kubelets request client certs via the **Certificates API** (now GA, `certificates.k8s.io/v1`), with `signerName` required.
- Default signers live in kube-controller-manager; approval can be auto or manual.
- cert-manager provides **experimental signers** (e.g., CA, Venafi) and is adding native support to **sign Kubernetes** `CertificateSigningRequest` objects using cert-manager issuers—opening more cluster bootstrap and rotation scenarios.

---

12) Putting It All Together

Beyond Ingress, cert-manager can automate certificates for:

- **Internal services** (server & client certs)
- **Pod-to-pod mTLS** (with or without a mesh)
- **DB client/server TLS**
- **Admission webhooks** (CA injection)
- **Service meshes** (Linkerd, Istio, OSM)
- **Cluster itself** (control plane and nodes via kubeadm + Certificates API)
- **Secretless delivery** via **CSI Driver** (node-local, in-memory keys)

Key practices:

- Prefer **short-lived certs** + automated renewal.

- Enforce **policy** (who can request what identities, key strength/algorithms).
- Choose **issuers** that match your trust model (public vs private).
- Consider **CSI** when Secrets are a compliance concern.

---

13) What's Next & How to Get Involved

- **Kubernetes Certificates API** support in cert-manager (sign CSR objects via any issuer).
- More **service mesh** and **issuer** integrations.
- Contributions welcome: code, docs, examples, labs, and issue triage.
- Join **Slack (`#cert-manager`)**, community meetings, and explore the labs (e.g., ping/pong mTLS) to get hands-on.

---

14) Summary

cert-manager started by making **Ingress TLS** effortless, but its power spans the entire cluster:

- Issue and renew certs for **apps, webhooks, meshes, control plane, and nodes**.
- Choose from **self-signed/CA**, **Vault**, **cloud CAs**, **Venafi**, or other **external issuers**.
- Go **secretless** with the **CSI Driver**, and scale with **policy** and **observability**.

> Make internal PKI **declarative, automated, and boring**—so teams can focus on shipping features, not babysitting certificates.